

IoT Project Report: Smart Storage Pod System

Inge Ackerman - 20251708

Nadine Allan - 20250828

Andrey Dyakov - 20251186

(IoT part Group 01 Project 01 – Collaborative Development)

1. Project Overview and Common Components

Our IoT project implements a complete smart bicycle storage pod system built on multiple ESP32 microcontrollers that work together to provide secure, modular, monitored, and user-friendly storage units.

The system is split into specialised nodes for educational purposes – each team member focused on different technologies and challenges while maintaining a unified architecture:

- **Main Unit** – central node per pod: collects environmental data (light, temperature, humidity), controls internal lighting, and manages pod-wide systems.
- **Entry Control Module** – one per entrance: handles user notification and access via multiple methods (physical button, mobile app command, RFID/NFC tags; QR code reading planned for next iteration).
- **Mechanical Lock Module** – secure servo-based claw with limit switches for securing bicycle in configurable predefined variations.

All active modules communicate bidirectionally with **Google Firebase Realtime Database** for state synchronisation, remote control, telemetry storage, and configuration. The design emphasises reliability, remote configurability, and extensibility.

1.1 Common Architectural Elements Across All Modules

Component	Implementation Details	Purpose
WiFi Connectivity	Multiple network support where implemented; robust reconnection; status LED feedback on some boards.	Reliable operation in varying environments.
Cloud Backend	Google Firebase Realtime Database • Project: varies slightly but same database • Authentication: legacy token (most modules) or email/password (environmental sensor prototype)	Centralised state, settings, telemetry, and remote monitoring/control.
Device Identification	Unique <code>device_id</code> or explicit path (e.g., <code>Pod_01_entrada_01</code> , <code>Pod_01_base_01</code> , <code>POD-A1</code>)	Clear addressing and scalability.
Heartbeat	Periodic timestamp updates (every 2–10 seconds, often configurable).	Cloud-side liveness monitoring.

Remote Configuration	Settings pulled periodically from Firebase; defaults ensured on boot.	Live tuning without reflashing.
Realtime Sync	Critical states (door/lock) read and written to Firebase; local actions pushed immediately.	Consistency between hardware and cloud/dashboard.
Team Collaboration	Modules developed in parallel by different team members to learn diverse techniques (button control, servo mechanics, light sensing, RFID, temperature sensing, Firebase auth methods).	Educational depth and modular design.

2. Module-Specific Descriptions

2.1 Door Entry Control – `unit_entrada.ino`

Purpose

Core user interface for pod access: physical push-button control with clear four-state visual feedback and cloud-synchronised door state.

Working physical prototype, button and notification LED are present on picture 1 at Chapter 7.

Key Features

- Four-state finite state machine (closed → opening → open → closing) with 5-second timed transitions.
- Button can toggle or abort ongoing processes (safety reverse).
- LED indicators: Green (closed), Blue (opening), Red (open), Yellow blinking (closing).
- Full bidirectional Firebase sync of `door_state` (pulled every 5 s, pushed on local change).
- Configurable heartbeat interval from cloud settings.

This module forms the baseline reliable entry control system.

2.2 Mechanical Claw/Lock Mechanism – `Pod_01_Claw_01.ino`

Purpose

Physical security: servo-driven claw with end-position limit switches and occupancy sensor.

Working physical prototype, Servo, Gears system, Limit Switch, Occupancy sensor and Claw are present on picture 2 at Chapter 7.

Key Features

- Configurable direction, stop position, and opening/closing durations via Firebase.
- Safety timeouts and immediate stop on limit switch hit.
- Usage counters (`open_cycles`, `occupancy`) for analytics.
- Robust defaults creation and periodic settings refresh.

2.3 Adaptive Lighting Control – `POD-Unit_Main_PhotoAI_v008.ino` (Main Unit Core)

Purpose

Primary environmental sensing and control node: maintains comfortable internal illumination using multiple photoresistors.

Working physical prototype, 4 Photoresistors and actuating LED are present on picture 3 at Chapter 7.

Key Features

- Four analog light sensors with configurable weights and rolling buffer averaging.
- Sophisticated control algorithm (proportional + small integral, threshold-based activation).
- Manual override mode.
- Comprehensive telemetry upload (individual averages, weighted average, brightness).
- Dynamic sampling rate and upload interval configurable from cloud.

This sketch runs on the central “mother” board that orchestrates pod-wide functions.

2.4 Temperature & Humidity Monitoring – `Temp_sketch_dec29a.ino` (Main Unit Core)

Purpose

Adds full environmental monitoring (temperature + humidity) to the main unit using a DHT11 sensor.

Connects only to Wi-Fi hotspot network.

Working physical prototype, Temperature & Humidity are present on picture 4 at Chapter 7.

Key Features

- Periodic readings (every 2 seconds) with NaN checking.
- Data packaged as JSON with timestamps (NTP-synchronised when available) and milliseconds fallback.
- Uses `pushJSON` to create time-series readings under `/pods/POD-A1/readings` (easily adaptable to the main pod path).
- Email/password Firebase authentication.

This module complements the existing light sensing to provide complete climate data for the pod interior and exterior weather for prediction with ML.model.

2.5 RFID/NFC Access Control Prototypes – `cicla_rfid_*` (Door Entry Control)

Purpose

Proof-of-concept and alternative access methods for the entry module using MFRC522 RFID reader.

Working physical prototype, RFID/NFC are present on picture 5 and 6 at Chapter 7.

Connects only to Wi-Fi hotspot network.

Two variants were developed:

2.5.1 Serial-Only Prototype – `cicla_rfid_serial.ino`

- Basic card detection and UID extraction.
- Formats UID as hyphenated uppercase hex string (e.g., `-89-BA-A9-C1`).
- Hard-coded access control: grants/denies based on known UIDs with serial feedback.
- Ideal for initial testing and debugging.

2.5.2 WiFi + Server Prototype – `cicla_rfid_wifi_connect_sketch_dec27a.ino`

- Same UID reading and formatting.
- On card detection, establishes HTTP connection to a local server/hotspot.
- Sends UID via GET request to PHP endpoint (`save_data.php`).
- Designed for integration with a backend authorisation system rather than hard-coded checks.

These prototypes demonstrate viable RFID integration paths for the entry module, allowing multiple access methods (physical button + RFID/NFC tags + app command). Full integration into the main `unit_entrada.ino` state machine is planned, along with **QR code scanning** in the next project iteration.

2.6 Reusable Firebase Helper – `PodFirebase.h` / `PodFirebase.cpp`

Purpose

Early attempt at encapsulating common Firebase operations into a class for cleaner code across devices.

Current Status

Provides wrappers for connection, path ensuring, int/timestamp operations. Not yet adopted in production sketches but represents planned refactoring direction.

3. Conclusion and Future Work

The collaborative project successfully delivers a functional smart storage pod prototype with:

- Secure mechanical locking
- Multi-method access control (button baseline + RFID prototypes)
- AI agent for adaptive internal lighting
- Full environmental monitoring (light + temperature/humidity)
- Cloud-connected remote control and analytics

The modular, team-split development allowed members to deeply explore different IoT techniques while maintaining architectural consistency through Firebase. The system is readily extensible to additional pods, more access methods (QR code planned), or enhanced analytics dashboards.

Future iterations will focus on full integration of RFID (and later QR) into the primary entry state machine, unified authentication approach, and possibly stream-based Firebase listening for lower latency.

4. ESP32 pins connection:

1. Door Entry Control – `unit_entrada.ino`

Used pins:

- GPIO12 → Push button (INPUT_PULLUP)
- GPIO18 → Green LED (closed)
- GPIO4 → Blue LED (opening)
- GPIO5 → Red LED (open)
- GPIO21 → Yellow LED (closing, blinking)

No SPI, no ADC, no PWM special requirements.

2. Mechanical Claw/Lock – `Pod_01_Claw_01.ino`

Used pins:

- GPIO13 → Servo signal (PWM)
- GPIO14 → Close limit switch (INPUT_PULLUP, NC)
- GPIO15 → Open limit switch (INPUT_PULLUP, NC)
- GPIO2 → WiFi status LED (built-in, blinking during connect)

Servo uses standard PWM via ESP32Servo library.

3. Adaptive Lighting (Main Unit) – POD-Unit_Main_PhotoAI_v008.ino

Used pins:

- GPIO34 → Photoresistor 0 (ADC1, input only)
- GPIO35 → Photoresistor 1 (ADC1, input only)
- GPIO32 → Photoresistor 2 (ADC1)
- GPIO33 → Photoresistor 3 (ADC1)
- GPIO25 → PWM LED output (brightness control)
- GPIO2 → WiFi status LED (built-in)

Heavy use of ADC1 channels.

4. Temperature & Humidity – Temp_sketch_dec29a.ino (Main Unit extension)

Used pins:

- GPIO4 → DHT11 data pin (one-wire protocol)

Single digital pin for DHT sensor.

5. RFID Prototypes – cicla_rfid_*.ino (Entry module extension)

Used pins:

- GPIO5 → MFRC522 SS/SDA (chip select)
- GPIO17 → MFRC522 RST (reset)

Hardware SPI (VSPI default):

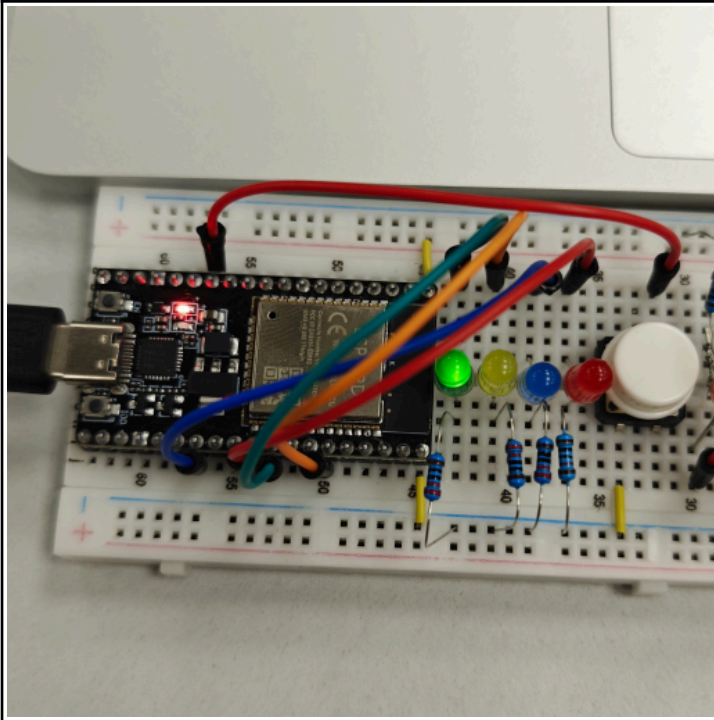
- GPIO23 → MOSI
- GPIO19 → MISO
- GPIO18 → SCK

Standard SPI connection for MFRC522 module.

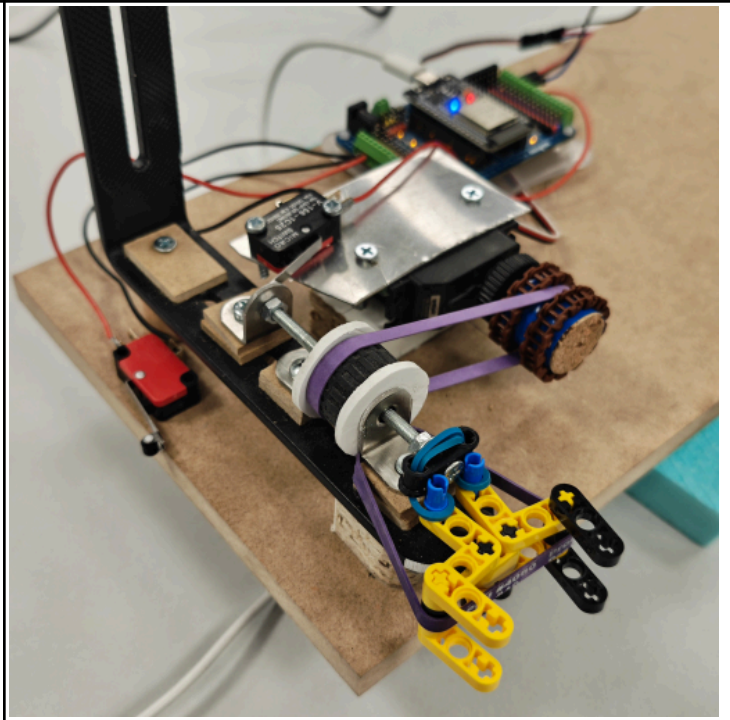
6 Pinouts conclusion

All modules use safe GPIOs that avoid flash pins (6–11), boot strapping conflicts, and input-only restrictions. No pin conflicts across modules when run on separate boards.

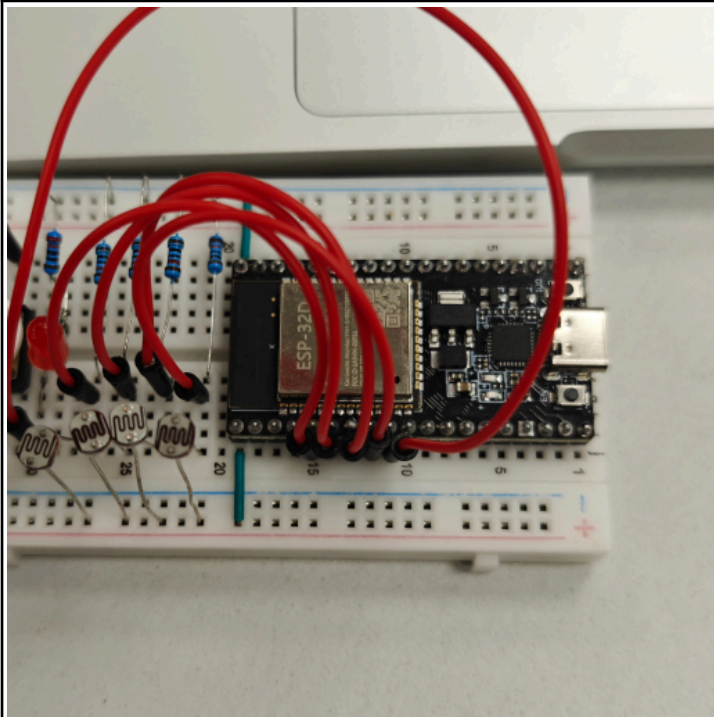
7 Images



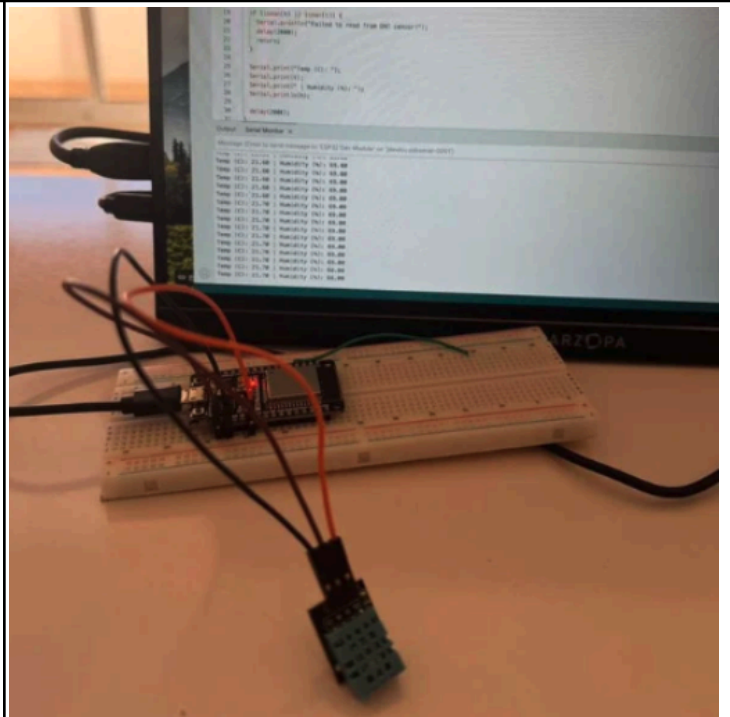
picture 1



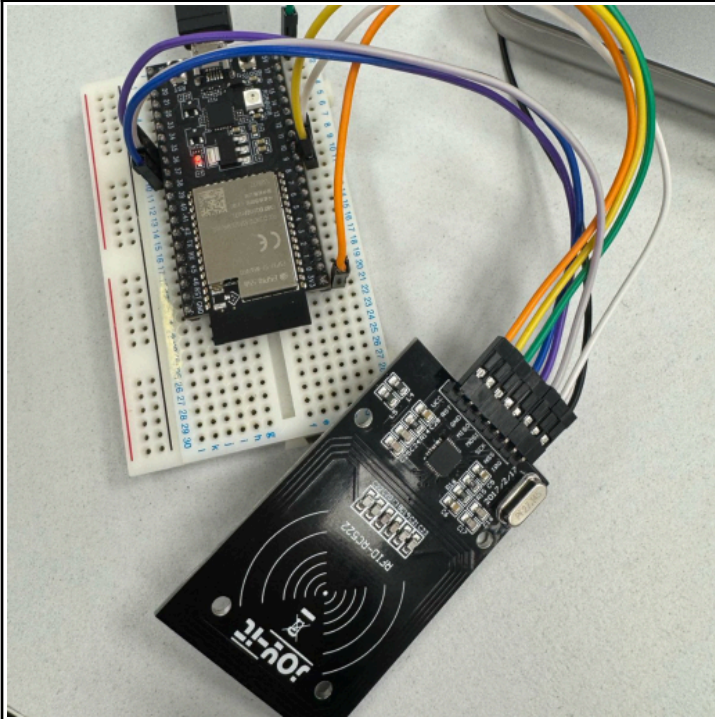
picture 2



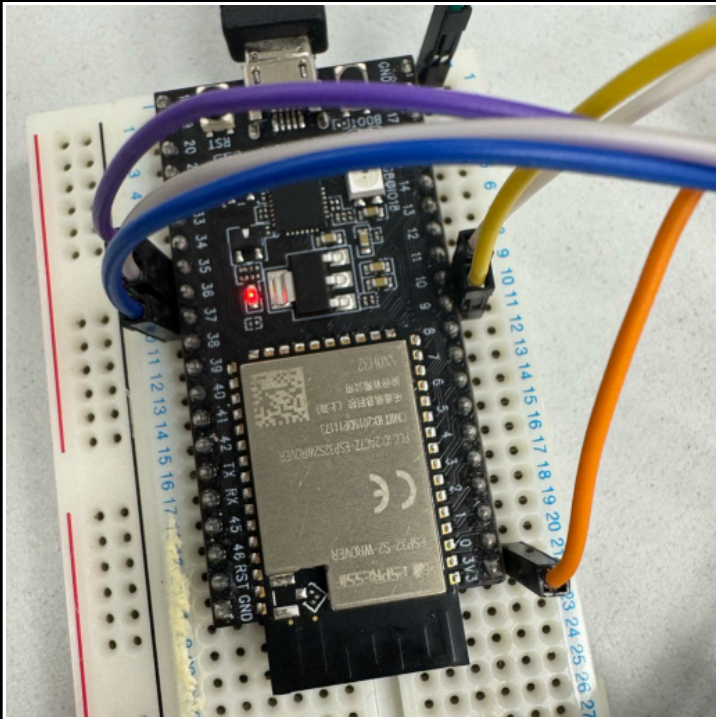
picture 3



picture 4



picture 5



picture 6